HE-IFD: Privacy-Preserving One-Shot Federated Distillation under Homomorphic Encryption

Halil İbrahim Kanpak, Sinem Sav, Alptekin Küpçü

***Abstract*—We present HE-IFD, a one-shot, privacy-preserving federated knowledge distillation framework in which all client contributions remain encrypted throughout server-side processing. Each client trains a local teacher model and extracts intermediate feature pairs at each block boundary, where a block is a contiguous group of layers (e.g., a residual stage) whose input and output are matched during distillation. These pairs are encrypted with homomorphic encryption (HE) and uploaded only once to the server. The server then distills these encrypted supervision signals into a full-depth HE-compatible student model without ever observing plaintext data, weights, or activations.**

Training an HE-compatible student with polynomial activations and no batch normalisation is unstable under standard end-to-end distillation. We address this with progressive stacking: block-by-block training where each block receives fresh ciphertexts, avoiding accumulation of HE multiplicative depth. Combined with client-side collaborative normalisation, magnitude-regularised loss, and student-input training, this procedure yields stable convergence. Under non-IID data partitions, the student aggregates specialised knowledge from all teachers, matching or exceeding the performance that each teacher achieves on its own local class distribution. This presents a strong participation incentive for every client, even when the student's global accuracy is lower than that of the best individual teacher.

***Index Terms*—Federated learning, knowledge distillation, homomorphic encryption, CKKS, polynomial activation, privacy-preserving machine learning**

I. INTRODUCTION

Federated Learning (FL) prevents raw data sharing among clients, but joint model training can still leak information through repeated gradient exchange [1], [2]. One-shot FL reduces communication overhead and privacy exposure by limiting communication to a single round, where intermediate gradient exchanges are not performed. However, if the transferred distillation targets are visible to the server in plaintext, the privacy risk remains [3].

Differential Privacy (DP) and secure aggregation partially address this risk, with their own limitations. Strong DP noise degrades distillation quality [4], while secure aggregation protects individual values without preventing the server from inspecting the aggregate [5]. Homomorphic encryption (HE) provides end-to-end cryptographic protection; yet direct iterative HE training of deep models is prohibitively expensive due to multiplicative depth constraints [6], [7], [8]. This motivates a distillation-based design that is both one-shot and HE-compatible. The main technical challenge is that HE computation supports only addition and multiplication over ciphertexts, whereas standard neural network operations, e.g., ReLU, batch normalisation, softmax, are non-polynomial. Recent work addresses the stability of polynomial activations for HE-compatible networks by deriving tighter ReLU approximations [9] and proposing boundary-loss and gradient-clipping strategies for deep polynomial training [10]. However, these methods target centralised inference or single-model training. Stable and efficient training of deep polynomial student models under encrypted federated supervision, where heterogeneous clients produce targets with incompatible magnitude distributions, remains an open problem.

For this reason, we focus on one-shot federated knowledge distillation under homomorphic encryption. In knowledge distillation, a compact *student* model learns by imitating the internal representations of a stronger pre-trained *teacher*, rather than training directly on labelled data. In our setting, each client acts as a teacher by training a local model on its private data and contributing encrypted supervision signals derived from that model in a single communication round. The server then uses these signals to train a privacy-preserving student entirely on ciphertexts, without ever accessing any plaintext data, weights, or activations.

A. Our Approach

We present **HE-IFD** (**H**omomorphic **E**ncryption-**I**ntermediate **F**eature **D**istillation), a framework in which individual client teacher outputs are protected during knowledge transfer and all subsequent server-side computation. The protocol operates in three phases:

1) **Local training.** Each client trains a standard teacher model (e.g., ResNet-18) on their private data partition, starting from a shared random initialisation.
2) **Encrypted upload.** Each client runs its teacher model on its own data and records the inputs and outputs at each layer group boundary. We refer to these groups as *blocks*. These input–output pairs tell the server what each block of the teacher produces, without revealing the raw data. Before encrypting, clients jointly agree on per-channel normalisation statistics so that features from different clients are on a comparable scale. The normalised pairs are then encrypted under Cheon–Kim–Kim–Song (CKKS) scheme and uploaded to the server in a single round; no further client interaction is needed.

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

3) **Server-side encrypted distillation.** The server trains a HE-compatible architecture in which non-polynomial operations such as ReLU and batch normalisation are replaced with polynomial equivalents that can be evaluated directly on ciphertexts (e.g., PolyResNet-18) for the student model on the server, block by block, using only encrypted inputs and targets. Student weights, activations, gradients, and loss values are all ciphertext throughout training.

The server never observes any individual teacher output, student weight, or intermediate activation in plaintext. It operates as a blind compute delegate that executes the distillation protocol on encrypted data. A key technical difficulty is training an HE-compatible network where non-polynomial operations (ReLU, batch normalisation) are replaced with polynomial equivalents that can be evaluated on ciphertexts, from encrypted intermediate supervision. Several compounding challenges make standard distillation approaches fail.

Polynomial magnitude growth is a structural property of the network class. A polynomial activation of degree d, composed across L blocks, yields an end-to-end map of degree d^L in its input. The induced upper bound on the activation magnitude grows doubly-exponentially in depth, regardless of how the polynomial coefficients are chosen, and this is what distinguishes polynomial deep networks from ReLU networks (whose composition is piecewise 1-Lipschitz). The federated setting amplifies the effect: heterogeneous clients induce different distributions on each block boundary, and the HE-compatible analogue of batch normalisation, a static per-channel affine map fixed before training, cannot adapt across distributions. Magnitude control is therefore a structural requirement for polynomial deep networks under federated encryption, not a numerical workaround.

Training and inference distributions diverge under block-wise composition. Decomposing the student into blocks and training each block independently confines the CKKS multiplicative depth to a single block (see Section III), but it also introduces a covariate shift across the composition: block k is trained on the distribution of teacher-produced inputs and evaluated on the distribution of student-produced inputs from the frozen prefix. The composed map cannot be consistent at inference unless this gap is closed. We close it through trainable per-channel affine bridges, initialised from the uploaded feature statistics, followed by a sequential refinement pass that exposes each block to its actual inference-time distribution. Both steps remain server-side and require no additional client communication.

The distillation loss must be scale-aware in magnitude and shape-aware in feature space. Standard mean squared error is not invariant to the sign asymmetry between ReLU and polynomial features and admits unbounded magnitude drift, which is then amplified through every frozen downstream block. The structural argument above forces the loss to control *shape* and *magnitude* separately. We use a channel-normalised mean-squared error to align the feature shape independently of scale, combined with a log-ratio penalty $(\log \sigma_{\hat{f}}/\sigma_{\tilde{f}})^2$ on the per-channel standard deviations. The log-ratio form is the natural choice for matching magnitudes: it is symmetric in over- and under-scaling and invariant to a uniform rescaling of the targets, properties that a plain ℓ_2 scale penalty does not satisfy.

Together, these three properties of polynomial deep networks under federated encryption (doubly-exponential magnitude growth, train-inference covariate shift across composed blocks, and the need for a scale-aware loss) determine the design of the protocol. Section IV develops each of them; Section V reports the resulting accuracy on CIFAR-10 and FashionMNIST across three architectures (ResNet-18, SimpleCNN, ViT-B/32), confirms that the framework generalises beyond a single model family, and shows that the encrypted student tracks the centralised plaintext Oracle and exceeds the mean individual teacher across all heterogeneity settings.

B. Contributions

Our contributions are as follows:

1) We propose, for the first time, progressive stacking for training HE-compatible student models fully under encryption: a block-by-block procedure with magnitude-regularised normalised MSE loss that achieves stable distillation of a full PolyResNet-18 student (11.17M parameters) from encrypted teacher features, where standard end-to-end distillation diverges.
2) We present a novel, fully encrypted one-shot distillation protocol in which clients upload encrypted per-block feature pairs in a single communication round and the server performs all distillation computation (forward pass, loss evaluation, backward pass, and weight update) entirely on ciphertexts, without exposing any plaintext data at any point during training.
3) We introduce client-side collaborative normalisation, a lightweight technique in which each client normalises its teacher outputs to zero mean and unit variance per channel before encryption, which eliminates magnitude incompatibility across heterogeneous teachers and prevents training divergence for $N \geq 4$ clients.
4) We provide a comprehensive empirical evaluation of HE-IFD across three architectures (ResNet-18, SimpleCNN, ViT-B/32), client counts $N \in \{1, 2, 4, 8, 16, 32\}$, and heterogeneity levels $\alpha \in \{0.1, 1.0\}$, including accuracy comparisons against differential privacy baselines, membership inference attack analysis, and communication and computation cost analysis.

II. Related Work and Motivation

A. Privacy Risks of Multi-Round Federated Learning

Federated learning (FL) [11] retains raw data locally but repeatedly transmits model updates with each round, presenting an attack surface. Shared gradients can be inverted to reconstruct training images [1], with recent attacks achieving exact reconstruction for batches up to $b \lesssim 25$ by exploiting ReLU-induced gradient sparsity [12]. Active and passive membership inference attacks exploit intermediate parameter updates [13], and GAN-based attacks reconstruct training data by exploiting evolving model dynamics across rounds [14]. Even with

secure aggregation, information accumulates: standard random user selection can leak individual user models within $O(N)$ rounds [2] N being the number of clients, and client-specific properties can be recovered from aggregated updates [15]. Consequently, repeated communication is inherently risky regardless of raw data sharing policies [16]. HE-IFD eliminates this attack surface by restricting communication to a single encrypted round.

B. Privacy Risks of Federated Distillation

Federated distillation is a class of FL methods in which clients train local teacher models and transfer knowledge to a server-side student by sharing model outputs such as logits, soft labels, or intermediate features, rather than gradients. Several protocols utilize this paradigm: FedMD [17] and DS-FL [18] have clients share softmax predictions on a shared public dataset, FedDF [19] distils an ensemble of heterogeneous client models into a single server model, and Cronus [20] adds robust aggregation via spectral filtering of client predictions. However, plaintext logits and features leak private information even without gradient access, posing a central risk to unencrypted federated distillation. Model confidence values inherently provide sufficient information for inversion attacks, enabling adversaries to reconstruct training data features directly [21].

Against FedMD-style distillation, the Paired-Logits Inversion attack [22] allows a malicious server to recover high-fidelity private images by exploiting the confidence gap between its predictions and those of the clients. Even without reconstructing individual samples, an honest-but-curious server can infer a client's label distribution solely from aggregated predictions [23], [24].

Membership inference attacks pose a distinct threat by revealing whether a specific sample was used for training. FD-Leaks [25] uses a malicious client's local model as a shadow model, while GradDiff and GradDrift [26] enable gradient-based inference by both servers and clients. More recent likelihood-ratio methods achieve stronger results: Co-operative LiRA and distillation-based LiRA variants [24] attain high true-positive rates at low false-positive rates against FedMD and DS-FL. Distillation itself amplifies these vulnerabilities. As an example, students can exhibit up to $+9.81\%$ higher MIA susceptibility than their teachers [27], and the GLiRA framework [28] shows that knowledge-distillation-based shadow models improve likelihood-ratio attacks over the original LiRA baseline [29]. Even transmitting hard labels instead of soft labels only mitigates rather than eliminates this risk, as discrete predictions still leak label distributions [30].

This diverse attack landscape demonstrates that plaintext knowledge transfer is fundamentally unsafe regardless of format. Only cryptographic protection eliminates these attack vectors entirely without degrading the distillation signal. HE-IFD encrypts all transferred knowledge under CKKS before upload, ensuring that the server never observes any plaintext supervision signal.

C. One-Shot Federated Distillation

One-shot federated learning [31], [32] restricts client-server communication to a single round and is the family of FL designs that share our problem statement. Among the plaintext distillation systems in this family, FedMD [17] and DS-FL [18] have clients transfer softmax predictions on a shared public dataset; FedDF [19] distils an ensemble of heterogeneous client models into a server model; Cronus [20] adds robust aggregation by spectrally filtering the client predictions; DENSE [33] uploads full plaintext models and trains a generator on the server; Co-Boosting [34] iteratively co-trains data and ensemble; and FuseFL [35] is architecturally closest to HE-IFD in that it also decomposes the student into blocks and trains them sequentially. None of these methods provide cryptographic protection: every client contribution reaches the server in plaintext and is therefore subject to the inversion, label-distribution, and membership-inference attacks reviewed in Section II-B. The distinguishing axis of HE-IFD relative to this family is privacy: the supervision signal is a ciphertext under collectively held keys, and the server's view is computationally indistinguishable from random regardless of the architectural similarity to FuseFL.

The smaller subset of one-shot FL systems that do offer formal privacy guarantees rely uniformly on differential privacy. FedKT [36] achieves $(\varepsilon, 0)$-DP through noisy voting on a public dataset; FedMD-NFDP [37] uses a noise-free sampling mechanism for (ε, δ)-DP; FedDiff [38] applies DP to diffusion-model training; and FedDM [39] supports optional DP on synthetic data. The structural cost of this approach is that one-shot settings consume the entire privacy budget in a single round, with no across-round amplification, so accuracy collapses toward random guessing at meaningful privacy levels ($\varepsilon \leq 1$) [37], [40]. The distinguishing axis of HE-IFD relative to this DP family is the type of guarantee: cryptographic and noise-free rather than statistical and noise-coupled to utility.

D. HE-Compatible Neural Network Design

Running neural networks on encrypted data requires that all operations reduce to addition and multiplication over ciphertexts. Standard operations such as ReLU, batch normalisation, and softmax are non-polynomial and cannot be evaluated under encryption. Prior work on *encrypted inference*, where a pre-trained model with known plaintext weights evaluates on encrypted inputs, replaces ReLU with learnable polynomial activations [41], [42] and absorbs batch normalisation parameters into preceding layers [43]. Recent centralised work has addressed the stability of training such polynomial networks: tighter ReLU approximations [9], boundary-loss and gradient-clipping schedules for deeper polynomial models [10], and reboot-style block-independent training for polynomial MLPs [44]. *Encrypted training* [45] is substantially harder than inference: both weights and data are ciphertexts, every learnable operation is ciphertext-by-ciphertext (consuming multiplicative levels), and the backward pass roughly doubles the depth of the forward pass. All of the works above operate on a single centralised model; HE-IFD is the first

to address stable polynomial-network training under fully encrypted, heterogeneous federated supervision, where teachers trained on different distributions produce target features with incompatible magnitude ranges.

E. Encrypted Federated Learning Systems

The works that HE-IFD is most directly compared with in Section V are encrypted federated learning systems. They share the cryptographic privacy goal but differ from HE-IFD on three axes: the unit that is encrypted (gradient updates vs. teacher features), the number of communication rounds, and where the multiplicative-depth budget is spent.

POSEIDON [45]. POSEIDON is the canonical multiparty-CKKS system for federated training; it supports the same threshold trust model as HE-IFD and shares its underlying multiparty-CKKS construction [46]. The unit that is encrypted is the per-round gradient: every client encrypts its gradient updates under the collective key, the server homomorphically aggregates them, and the result is broadcast back. The protocol is therefore inherently multi-round, and each round consumes the full forward+backward depth of the model under encryption, requiring bootstrapping to be invoked across rounds. HE-IFD differs on every axis: a single round, encryption of intermediate teacher features rather than gradients, and a depth budget that is confined to a single block of the student because each block is trained on fresh ciphertexts.

BatchCrypt [6]. BatchCrypt instantiates the same multi-round encrypted-gradient pattern with batched Paillier encryption rather than CKKS. The scheme is additively homomorphic, which is sufficient for FedAvg-style gradient aggregation but rules out the polynomial-network training that HE-IFD performs server-side. The relevant axis of comparison in our experiments is communication cost: BatchCrypt's per-round cost is dominated by encrypted gradients, scales linearly with the number of rounds, and gives the curve a slope similar to encrypted POSEIDON despite the different scheme.

FedSHE [47]. A more recent CKKS-based federated learning system that introduces adaptive packing strategies to compress the per-round encrypted-gradient upload, reducing the constant in front of the multi-round communication curve. FedSHE remains structurally multi-round and encrypts gradient updates rather than supervision signals; HE-IFD is orthogonal to this line of work in that it eliminates the multi-round pattern entirely rather than optimising its constants. The two designs are complementary: an adaptive-packing scheme analogous to FedSHE's would also reduce the size of HE-IFD's single ciphertext upload.

In the same broader category, Vanilla FedAvg [11] is the unencrypted multi-round baseline used in the communication-cost comparison, and FedML-HE [7] and CURE [8] are further multi-round encrypted-FL systems we cite for context but do not include as primary curves.

Motivation for One-Shot Communication and Homomorphic Encryption. One-shot federated learning [31], [32] limits client–server communication to a single round, eliminating the iterative exposure that facilitates multi-round attacks. Knowledge distillation suits this setting naturally: clients train local teachers and transfer knowledge to a server-side student in a single step. However, data heterogeneity causes inconsistent teacher logits, and as shown above, even this single round of plaintext transfer remains exploitable by the server. Encrypting the transferred knowledge is therefore necessary, not optional.

Homomorphic encryption (HE) [48] provides end-to-end cryptographic protection without injecting noise, avoiding the utility–privacy tradeoff of DP. However, applying HE to standard iterative training is structurally expensive. The systems reviewed in Section II-E (POSEIDON, BatchCrypt, FedSHE) all encrypt per-round gradient updates and therefore exhaust the multiplicative-depth budget of CKKS [48] (further discussed in Section III) in every round; bootstrapping [49] refreshes the budget but at substantial latency. One-shot settings resolve this tension: ciphertexts are used once, and block-independent training with fresh ciphertexts confines the depth budget to a single block of the student rather than the full network. This principle was recently validated for encrypted MLP training [44]. HE-IFD extends it to federated one-shot distillation over heterogeneous clients.

DP partially addresses plaintext leakage but degrades the distillation signal. Applying local DP causes severe accuracy drops (up to 70% on CIFAR-100 at $\epsilon = 0.1$ [40]), and one-shot settings consume the entire privacy budget immediately, precluding amplification across rounds. At meaningful privacy levels ($\epsilon \leq 1$), DP reduces FedMD to near random-guessing accuracy [37]. HE avoids this tradeoff entirely.

III. Preliminaries: CKKS Homomorphic Encryption

The Cheon–Kim–Kim–Song (CKKS) scheme [48] is a homomorphic encryption scheme that supports approximate arithmetic over vectors of floating point numbers. Unlike exact HE schemes, CKKS is designed for machine learning workloads where small numerical errors are acceptable.

Slots and batching. A CKKS ciphertext encrypts a vector of up to $\mathcal{N}/2$ real numbers simultaneously, where $\mathcal{N}$ is the polynomial modulus degree (a power of two). Each position in this vector is called a *slot*. For example, with $\mathcal{N} = 8{,}192$, a single ciphertext holds 4,096 slots. This SIMD (single instruction, multiple data) packing means that one encrypted operation applies simultaneously to all slots, enabling efficient batched computation over large feature maps.

Multiplicative depth. Each ciphertext carries a finite *multiplicative depth budget* which is the maximum number of ciphertext-by-ciphertext multiplications that can be performed before the noise floor exceeds the signal. Each such multiplication consumes one multiplicative *level*. Addition and ciphertext-by-plaintext multiplication are budget-free. Once the entire budget is exhausted, i.e., multiplicative levels are consumed, a *bootstrapping* operation can refresh the budget, but at substantial computational cost ($\sim$20 seconds per ciphertext). HE-IFD minimises bootstrapping by training each block independently on fresh ciphertexts (Section IV-A2), making the per-block depth to $\sim$12 levels. Bootstrapping is used only during the cascade refinement step that composes blocks into the full model.

Security. CKKS security relies on the Ring Learning with Errors (RLWE) hardness assumption [50]. Under this assump-

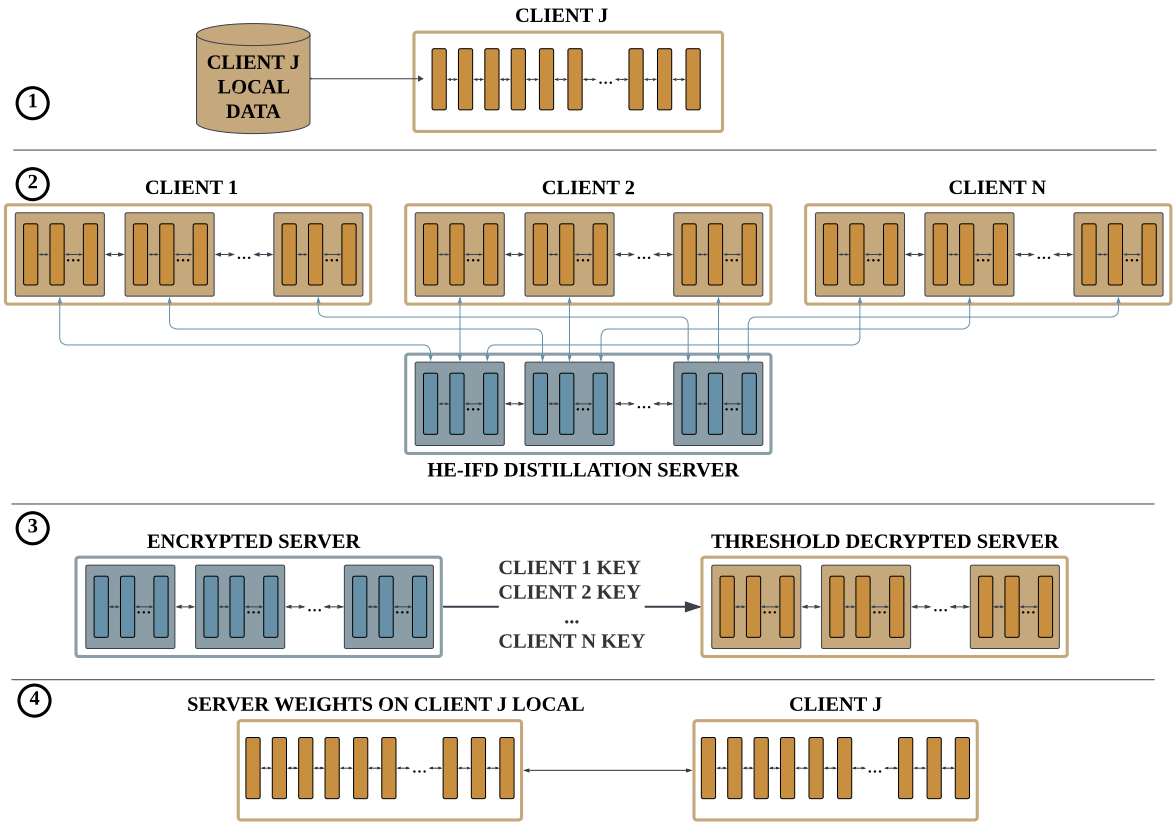

Fig. 1. Overview of HE-IFD. ① **Client-side training.** Each client independently trains a teacher model on its own private dataset. No data leaves the client device. ② **Encrypted upload and server distillation.** Every client extracts intermediate feature representations at each block boundary of its teacher, normalises them per channel, encrypts them under the CKKS homomorphic encryption scheme, and uploads the ciphertexts to the server in a single communication round. The server pools encrypted features from all clients and trains a student model block by block, operating entirely on ciphertexts. The server never observes any plaintext data, weights, or intermediate values. ③ **Threshold decryption.** The server distributes the encrypted student model to all clients. Each client applies its secret key share locally to decrypt the model on its own device. No single client can decrypt alone, and the server never sees plaintext weights. ④ **Local model assembly.** Each client holds the decrypted student model in plaintext and may optionally fine-tune it on local data to adapt to its own distribution, with no further communication required.

tion, a ciphertext is computationally indistinguishable from a random ring element (IND-CPA security): an adversary who observes only ciphertexts learns nothing about the encrypted values. All intermediate results of homomorphic computation, i.e., activations, gradients, loss values, remain ciphertexts and inherit this guarantee.

IV. METHODOLOGY

A. HE-IFD: One-Shot Distillation Framework

This section presents the HE-IFD protocol in full generality. The framework applies to any architecture that admits a block decomposition with HE-compatible operations; architecture-specific instantiations are deferred to Section IV-D.

System Model. We consider a cross-silo federated learning setting with N clients and a central server. Each client i holds a private dataset $\mathcal{D}_i$ that is never shared or transmitted. Clients wish to collaboratively train a shared student model by transferring knowledge from their locally trained teacher models to the server, without exposing their private data, including raw training data and any intermediate representations. Communication between clients and the server is restricted to a single round: each client sends a set of encrypted messages derived from its local data and model, after which the server performs all remaining computation independently with no further client interaction. An optional release phase at the end allows clients to collectively obtain the trained model.

Threat Model. We assume a semi-honest (honest-but-curious) server that follows the protocol faithfully but may attempt to infer private information from everything it observes. The adversary's goal is to infer information about a client's private dataset $\mathcal{D}_i$ or the client's local model, including individual samples, label distributions, or model weights, from the messages it receives during training. We further assume that the server may collude with up to $N - 1$ clients; privacy of the remaining honest client's input is guaranteed by the threshold structure of the multiparty CKKS scheme described in Section IV-A3, which requires the joint participation of all N clients to decrypt any ciphertext.

Given this system and threat model, sending plaintext teacher outputs to the server is insufficient. Because, as discussed in Section II-B, even a single round of plaintext knowledge transfer exposes clients to inference and inversion attacks. We therefore instantiate the knowledge transfer channel using the multiparty CKKS HE scheme [48], [46] —

the same construction implemented in Lattigo [51] — which lets the server perform all distillation computation directly on ciphertexts under a public key whose corresponding secret key is held in shares across the N clients.

Overview of HE-IFD. HE-IFD proceeds in four phases, illustrated in Figure 1 and formalised in Algorithm 1. In Phase 1, each client independently trains a teacher model on its private data. In Phase 2, clients extract intermediate features at each block boundary of the teacher, encrypt them under CKKS, and upload to the server in a single communication round, which is the only client-to-server interaction in the entire protocol. The server then trains an HE-compatible student model block by block on the pooled encrypted features. In Phase 3, clients collaboratively perform threshold decryption to obtain the trained student in plaintext. In Phase 4, each client receives the decrypted model (via joint threshold decryption) and may fine-tune it on local data to adapt to its own distribution.

The server never sees any plaintext data, model weights, or intermediate computations. All training happens on encrypted ciphertexts. Each block is trained independently with fresh ciphertexts, so multiplicative levels do not accumulate across blocks.

1) Phase 1: Client-Side Training: Each client i trains a teacher model T_i on its private data $\mathcal{D}_i$ using standard supervised learning. All clients start from the same shared random weight initialisation. This phase is fully local, and no data leaves the client device.

2) Phase 2: Encrypted Upload and Server-Side Distillation: Phase 2 has two parts: first, clients extract and upload encrypted features (one-shot); then, the server trains a student model entirely on these encrypted features.

Feature extraction. Each client runs its trained teacher on local data and extracts input-output feature pairs $(\mathbf{x}_k, \mathbf{y}_k)$ at every block boundary $k \in \{0, \ldots, K\}$, where block boundaries match the student architecture (Section IV-D). Clients use their own local data, not a shared auxiliary dataset. This is important: each teacher produces high-confidence outputs for the classes it has seen, and using local data preserves this specialisation rather than diluting it with uncertain predictions on unfamiliar classes.

Normalisation. Consistent normalisation across clients is essential for stable encrypted training. We adopt a collaborative normalisation approach where each client computes local statistics and the server aggregates them on encrypted data, following the privacy-preserving federated normalisation framework of [52]. Before encryption, each client normalises features per channel to zero mean and unit variance:

$$\mu_c^{(k,i)} = \mathbb{E}_{x \in \mathcal{D}_i}[y_{k,c}(x)], \quad \sigma_c^{(k,i)} = \mathrm{Std}_{x \in \mathcal{D}_i}[y_{k,c}(x)] \tag{1}$$

$$\tilde{y}_{k,c} = \frac{y_{k,c} - \mu_c^{(k,i)}}{\sigma_c^{(k,i)}} \tag{2}$$

where $y_{k,c}$ is the output of channel c at block boundary k, $\mathbb{E}[\cdot]$ and $\mathrm{Std}[\cdot]$ denote the mean and standard deviation over all samples in client i's local dataset $\mathcal{D}_i$, and C is the total number of channels at that block boundary. This is not optional since, without it, teachers trained on different distributions produce features with incompatible magnitude ranges, and the server's training diverges when pooling them.

Algorithm 1 HE-IFD: HE-Based Intermediate Feature Distillation

Require: N clients with private datasets $\mathcal{D}_1, \ldots, \mathcal{D}_N$; student architecture S with $K{+}1$ blocks
Ensure: Trained student model S

— Phase 1: Client-Side Training (parallel, local) —
for each client $i = 1, \ldots, N$ **in parallel do**
  Train teacher T_i on $\mathcal{D}_i$ via standard SGD
end for

— Phase 2: Encrypted Upload + Server Distillation —
for each client $i = 1, \ldots, N$ **in parallel do**
  for each block $k = 0, \ldots, K$ **do**
    Extract feature pairs $(\mathbf{x}_k^{(i,j)}, \mathbf{y}_k^{(i,j)})$ by running T_i on $\mathcal{D}_i$
    Normalise per channel: $\tilde{\mathbf{y}}_k \leftarrow (\mathbf{y}_k - \mu)/\sigma$ {Eq. 2}
  end for
  Encrypt all pairs under CKKS; upload $\{[\![\mathbf{x}_k]\!], [\![\tilde{\mathbf{y}}_k]\!]\}$ to server {one-shot}
end for

Server-side (all computation on encrypted data):
Initialise student S with near-linear polynomial activations
for block $k = 0, \ldots, K$ **do**
  Pool encrypted pairs from all N clients for block k; shuffle
  Set depth-dependent hyperparameters: λ_k, η_k {deeper blocks: stronger reg., lower LR}
  for epoch $= 1, \ldots, E$ **do**
    for each mini-batch $([\![\mathbf{x}]\!], [\![\tilde{\mathbf{y}}_k]\!])$ **do**
      $\hat{\mathbf{y}} \leftarrow S_{\text{block}[k]}([\![\mathbf{x}]\!])$ {encrypted forward pass}
      $\mathcal{L} \leftarrow \mathcal{L}_{\text{MagReg}}(\hat{\mathbf{y}}, [\![\tilde{\mathbf{y}}_k]\!])$ {Eq. 4}
      Update $S_{\text{block}[k]}$ via encrypted gradient descent
    end for
  end for
  Freeze $S_{\text{block}[k]}$; initialise bridge $\mathbf{b}_k$ from feature statistics
end for
Refine blocks sequentially with bridges on uploaded features (R rounds)

— Phase 3: Threshold Decryption —
All N clients contribute key shares $\rightarrow$ decrypt S to plaintext

— Phase 4: Client-Side Model Assembly —
for each client $i = 1, \ldots, N$ **do**
  Receive decrypted student S
  (Optional) Fine-tune S on local data $\mathcal{D}_i$ with cross-entropy loss
end for

Encrypted upload. Clients encrypt normalised features under CKKS and upload to the server. This is a single communication round and no further client interaction is needed. Normalisation statistics ($2C$ values per block per client, where C is the number of channels)) may be sent in plaintext since they are aggregate quantities that reveal unimportant information about individual samples.

Server-side distillation. The server trains the student model block by block on the pooled encrypted features from all clients. For each block k, the server receives fresh encrypted input-output pairs $\left([\![\mathbf{x}_k^{(i,j)}]\!], [\![\tilde{\mathbf{y}}_k^{(i,j)}]\!]\right)$ from every client i. It pools and shuffles them, then trains block k via encrypted gradient descent. Upon convergence, block k is frozen and the server moves to block $k{+}1$ using fresh ciphertexts.

This is the key design choice: each block trains on its own fresh ciphertexts, so CKKS multiplicative levels never accumulate across blocks. The total depth per block depends only on that block's structure, regardless of network depth.

Unlike federated averaging, the server does not aggregate teacher outputs. Each client's features provide independent gradient steps that accumulate into shared student weights. A client who specialises in certain classes contributes strong supervision for those classes, with no signal dilution.

Train-inference distribution gap. When block k is trained on teacher features (ReLU-based: non-negative, sparse) but receives student features at inference (polynomial: signed, dense), a distribution gap arises. Empirically, composed accuracy collapses to $\sim$10% without addressing this.

We close this gap within the one-shot protocol by having each block trained in two stages. In the first stage, the block is trained on teacher features to learn the correct mapping. In the second stage (refinement), the block is fine-tuned on the uploaded encrypted data with a per-channel affine *TrainableBridge* $\mathbf{b}_k(\mathbf{h}) = \boldsymbol{\gamma}_k \odot \mathbf{h} + \boldsymbol{\beta}_k$ that compensates for the distribution shift. Bridge parameters are initialised from feature statistics and co-trained with block weights during refinement.

Loss Function. Each block is trained with a magnitude-regularised normalised mean squared error loss. Student and teacher features are normalised per channel to zero mean and unit variance before computing the error:

$$\mathcal{L}_{\text{NormMSE}} = \frac{1}{C}\sum_{c=1}^{C} \text{MSE}\left(\frac{\hat{f}_c - \mu_{\hat{f}_c}}{\sigma_{\hat{f}_c}}, \frac{\tilde{f}_c - \mu_{\tilde{f}_c}}{\sigma_{\tilde{f}_c}}\right) \quad (3)$$

where $\hat{f}_c$ and $\tilde{f}_c$ are the student prediction and normalised teacher target for channel c. This scale-invariant loss handles the distributional mismatch between student activations (signed, potentially large) and teacher activations.

Magnitude regularisation. The normalised loss alone is scale-invariant and permits the student to grow magnitudes without penalty. Since subsequent frozen polynomial blocks amplify any scale drift, we add a log-ratio penalty:

$$\mathcal{L}_{\text{MagReg}} = \mathcal{L}_{\text{NormMSE}} + \lambda \cdot \frac{1}{C}\sum_{c=1}^{C}\left(\log\frac{\sigma_{\hat{f}_c}}{\sigma_{\tilde{f}_c}}\right)^2 \quad (4)$$

The log-ratio form is symmetric (penalising student magnitudes that are too large or too small equally) and scale-free (proportional to the multiplicative mismatch). The regularisation weight λ scales with block depth: stronger regularisation is applied to deeper blocks where magnitude accumulation from the frozen chain is more severe.

MSE on logits. For the head block (block K), which outputs class logits, we use plain MSE without normalisation or magnitude regularisation, since logits are low-dimensional and do not exhibit the spatial channel structure of intermediate features. We use MSE rather than KL divergence because softmax is not a polynomial operation and cannot be evaluated under CKKS encryption. MSE on raw logits is HE-compatible because the squaring operation requires one ciphertext-ciphertext multiplication, and the remaining subtraction and averaging are level-free.

3) Phase 3: Threshold Decryption: We instantiate the encrypted channel of HE-IFD with the multiparty CKKS protocol of [46], available in the open-source Lattigo library [51]. We describe how the protocol is used in HE-IFD; the underlying construction is unchanged from the cited references and we treat it as a black box.

Setup (before Phase 1). The N clients run a distributed key generation (DKG) protocol with no trusted dealer. Each client i samples a secret-key share sk_i from the CKKS secret-key distribution and contributes a corresponding share to the protocol. The DKG output is (i) a collective public key pk usable by everyone for encryption, (ii) collective evaluation and rotation keys usable by the server for homomorphic computation, and (iii) the secret-key shares $\{\mathsf{sk}_i\}_{i=1}^{N}$, kept privately by the clients. The full secret key $\mathsf{sk} = \sum_{i=1}^{N} \mathsf{sk}_i$ is never assembled at any single party at any point in the protocol.

Encryption (Phase 2). Every client encrypts its normalised feature pairs (Section IV-A2) under the collective public key pk and uploads the ciphertexts to the server. From the server's perspective, the resulting ciphertexts behave exactly as ordinary single-key CKKS ciphertexts: the server applies homomorphic additions, ciphertext-by-plaintext multiplications, ciphertext-by-ciphertext multiplications (using the collective relinearisation key), and slot rotations (using the collective rotation keys) in the usual way. No client interaction is required during this phase.

Collective decryption (Phase 3). To release the trained student in plaintext, the server initiates a single round of *collective key-switching* [46]: it broadcasts the encrypted student weights $[\![\mathbf{W}]\!]$ to the clients, and each client i contributes a key-switching share that re-encrypts $[\![\mathbf{W}]\!]$ from the collective key pk to a target key pk' (e.g., the all-zero key, which yields plaintext, or a fresh public key whose corresponding secret is held by a designated recipient). Every client's share is computed locally from sk_i and added a small smudging noise to mask its individual contribution. Combining all N shares yields the re-encrypted ciphertext, from which $\mathbf{W}$ can be decrypted; combining any strict subset of $N-1$ shares yields a uniformly random ring element under the RLWE assumption [50]. If this phase is skipped, the student stays encrypted and the server can serve encrypted inference directly.

Trust assumption. Privacy of every client's input holds as long as at least one of the N clients is honest. This is strictly stronger than the trust assumption of single-key CKKS deployments, which require a trusted decryption authority, and matches the threat model of Section IV-A: even if the server colludes with $N-1$ clients, the joint view consists of ciphertexts under a key whose remaining share is held by the honest client and is therefore computationally indistinguishable from random.

4) Phase 4: Client-Side Model Assembly and Fine-Tuning: After local decryption, each client holds a copy of the trained student model in plaintext. Clients may fine-tune this model on their own private data using standard cross-entropy loss. This step requires no further communication with the server or other clients. Fine-tuning is particularly useful under non-IID settings, where the global student may underperform on a

client's local distribution. We evaluate the accuracy impact of local fine-tuning in Section V.

B. Training Stability

As established in Section I, the composition of degree-d polynomial activations across L blocks yields an end-to-end map of degree d^L, whose magnitude bound grows doubly-exponentially in depth. This is a property of the function class rather than of any particular implementation, and it must be controlled by a combination of mechanisms that act on different parts of the protocol. Concretely:

- **Client-side normalisation** (Eq. 2): ensures features from different clients have compatible magnitude ranges.
- **Magnitude-regularised loss** (Eq. 4): penalises scale drift between student and teacher outputs.
- **Gradient clipping**: prevents large updates from the quadratic term.
- **Depth-dependent hyperparameters**: deeper blocks use stronger regularisation and lower learning rates.
- **Per-channel affine bridges**: compensate for the absence of batch normalisation at block boundaries.

Section V provides an ablation study quantifying the impact of each mechanism.

C. Privacy Analysis

During Phase 2, the server sees only CKKS-encrypted data: teacher block-boundary feature pairs, basic metadata such as tensor shapes and client count, and optional plaintext per-channel normalisation statistics. Raw client data and teacher model weights never leave the client. All server-side computations, i.e., forward passes, loss evaluation, gradient descent, and weight updates, are performed homomorphically on ciphertexts; student weights, intermediate activations, gradients, and loss values remain encrypted throughout Phase 2.

Under the Ring Learning with Errors (RLWE) assumption (with security parameter $\lambda \geq 128$), our protocol guarantees that the server learns no private information about individual client data during training. This relies on the IND-CPA security of the CKKS scheme [48], which means encrypted data cannot be distinguished from random noise. Because all homomorphic operations simply map ciphertexts to other ciphertexts, every intermediate calculation stays secure. As a result, the server's entire view of the training process can be simulated using only public parameters and metadata, proving no data is leaked. If clients choose to send their normalisation statistics in plaintext, these are aggregate quantities computed over the full local dataset and carry negligible information about any individual sample. For maximum security, these statistics can also be encrypted following the protocols in [52].

D. Architecture-Specific Instantiations

The HE-IFD framework described in Section IV-A applies to any neural network that can be decomposed into blocks with HE-compatible operations. This section describes how we instantiate the framework for convolutional networks and vision transformers. The same principles extend to language models, which we discuss briefly for completeness.

1) Convolutional Networks: **PolyResNet-18.** The primary convolutional instantiation is PolyResNet-18, which is structurally identical to the ResNet-18 teacher but with all HE-incompatible operations replaced. This structural correspondence enables block-level feature matching between teacher and student. We summarize the replacements below:

ReLU $\rightarrow$ PolyAct. ReLU ($\max(0, x)$) has no efficient polynomial representation under CKKS. We replace it with a learnable degree-2 polynomial:

$$\text{PolyAct}(x) = ax^2 + bx + c \tag{5}$$

initialised at $a = 0.3$, $b = 0.7$, $c = 0$, so that for normalised inputs the activation behaves approximately linearly, avoiding early magnitude divergence from the quadratic term. This near-linear initialisation strategy is consistent with prior work on stable polynomial-activation training [41], [42]. The coefficients are learnable and adapt during training; after training, they are frozen as plaintext constants. Under CKKS, x^2 is one ciphertext-ciphertext multiplication (1 multiplicative level). The remaining terms (ax^2, bx, c) involve only plaintext-ciphertext operations (0 levels). Thus, the total cost is 1 multiplicative level per activation.

BatchNorm $\rightarrow$ ChannelScale. Batch normalisation computes a normalised output $\hat{x} = (x - \mu)/\sigma$ followed by a learnable affine transform. Under CKKS, the division x/σ has no direct equivalent and must be approximated via iterative methods such as Goldschmidt's algorithm, which requires approximately 10 multiplicative levels per invocation. Since batch statistics are fixed after training, the normalisation can be folded into the affine parameters, leaving only the learnable scale and bias. We therefore replace BatchNorm with ChannelScale:

$$\text{ChannelScale}(x) = \gamma \odot x + \beta \tag{6}$$

with learnable γ (initialised to 1) and β (initialised to 0). Since both are known plaintext at inference, this costs 0 multiplicative levels.

MaxPool $\rightarrow$ Identity. Max pooling is a comparison operation that requires non-polynomial approximations with prohibitive multiplicative depth under CKKS. For CIFAR-10 (32×32 input), the initial max pool in standard ResNet-18 is replaced by an identity operation. Spatial downsampling is handled by stride-2 convolutions in deeper layers.

PolyBasicBlock. Each PolyBasicBlock, the residual building block of PolyResNet-18, follows the dataflow:

$$x \xrightarrow{\text{conv}_1} \xrightarrow{\text{CS}_1} \xrightarrow{\text{PolyAct}_1} \xrightarrow{\text{conv}_2} \xrightarrow{\text{CS}_2} \xrightarrow{+\,\text{identity}} \xrightarrow{\text{PolyAct}_2} y \tag{7}$$

where CS denotes ChannelScale and $+$ identity is the residual addition. Convolutions, ChannelScale, and the residual addition do not consume any levels. The two PolyAct invocations consume 1 level each, for a total of 2 multiplicative levels per block.

Block boundaries. We define six blocks for ResNet-18: Block 0 (stem) takes raw images as input and outputs the result after the first convolution, ChannelScale, and PolyAct. Blocks 1 through 4 correspond to the four residual layer groups, each containing two PolyBasicBlocks. Block 5 (head)

processes the final layer's output into class logits via average pooling and a fully connected layer.

Compact variants and simpler architectures. We also evaluate PolyResNet-10 (one block per layer instead of two, approximately 5.6 million parameters) and PolyResNet-8 (three layers instead of four, approximately 1.5 million parameters) to study the effect of model capacity. For lightweight settings, we instantiate HEStudentCNN, a plain convolutional network with configurable channel widths, polynomial activations, and per-channel affine normalisation (no residual connections). These compact architectures demonstrate that the framework generalises beyond the ResNet family.

2) Transformer and Vision Transformer Architectures: HE-IFD naturally extends to transformer-based models through logit-level KL distillation rather than block-level feature matching. Transformer attention mechanisms create inter-layer dependencies that make intermediate feature matching less effective, but logit-level distillation requires only that the student match the teacher's final output distribution.

A key advantage of HE-IFD for transformers is its compatibility with pre-trained models. Rather than training from scratch, each client fine-tunes a pre-trained model (e.g., ViT-B/32 pre-trained on ImageNet) on their local data in Phase 1. This makes the framework practical for large models that would be infeasible to train from scratch on small client datasets. After encrypted distillation (Phase 2) and threshold decryption (Phase 3), clients receive the student model and may further fine-tune it locally in Phase 4 using standard (non-polynomial) activations, recovering any accuracy lost from HE-compatible replacements.

Vision Transformer (ViT). In Phase 1, each client takes a ViT-B/32 model pre-trained on ImageNet-1K (88M parameters) and fine-tunes it on their local private data partition to produce a teacher. In Phase 2, clients extract logits from their fine-tuned teachers, encrypt them, and upload to the server. The server distils these into an HE-compatible student where GELU activations are replaced with PolyGELU (degree-4 polynomial) and LayerNorm with AffineNorm. We evaluate on CIFAR-10 and FashionMNIST.

HE-compatible replacements for transformers. Two operations in standard transformers are incompatible with CKKS: the GELU activation function and LayerNorm. We replace GELU with a degree-4 polynomial approximation (PolyGELU) fitted to match GELU over the range $[-5, 5]$. LayerNorm, which requires a division by the standard deviation, is replaced by AffineNorm: a per-element learnable scale and bias without any running statistics or normalisation division. These replacements are analogous to the PolyAct and ChannelScale substitutions in the convolutional setting.

HE compliant student training. To ease optimisation, we train transformer students in two phases. In the warmup phase, the student retains the standard GELU and LayerNorm activations, and is trained with KL divergence loss for several epochs to converge to a good parameter region. In the conversion phase, all GELU activations are replaced with PolyGELU and all LayerNorm layers are replaced with AffineNorm, and training continues with a reduced learning rate to adapt to the polynomial activation landscape. This two-phase strategy avoids the difficulty of training with polynomial activations from scratch, which we found to be unstable for transformer architectures.

Memory efficiency. For models with large output spaces, teacher logit tensors can be very large. We store averaged logits in half precision (float16), which halves memory usage with negligible impact on distillation quality. Logit averaging across clients is performed via in-place accumulation rather than stacking, reducing peak memory from three times the logit tensor size to two times.

E. Communication and Computation Complexity

1) CKKS Multiplication Budget: Each ciphertext-ciphertext (CT×CT) multiplication consumes one multiplicative level. Ciphertext-plaintext (CT×PT) multiplications and all additions are level-free. Once a ciphertext's levels are exhausted, bootstrapping can refresh them, but at high computational cost.

On the server, **both the student weights and the client features are encrypted ciphertexts**. The server never holds plaintext weights or data. Every learnable operation (convolution, affine scaling, fully connected layers) is CT×CT and consumes 1 level. Polynomial activations of degree d consume $\lceil \log_2 d \rceil$ levels (e.g., degree-2 costs 1 level for x^2, degree-4 costs 2 levels for $x^2 \to x^4$). Additions (residual connections, bias terms) and plaintext-scalar operations (average pooling) are level-free, consuming no multiplicative depth.

Per-block training depth. HE-IFD trains each block independently with fresh ciphertexts, so levels never accumulate across blocks during Phase 2a. The depth consumed per block depends only on that block's internal structure. For the architectures we evaluate, a single block's forward pass consumes 4–16 levels depending on complexity. Adding loss computation (1 level for MSE) and the backward pass, the total per-block training depth fits within the budget provided by ring dimension $\log \mathcal{N} = 15$ ($\sim$18 levels at 128-bit security). Phase 2a therefore, requires no bootstrapping. The cascade refinement in Phase 2b composes multiple blocks and exceeds the per-block budget, requiring bootstrapping to refresh levels between blocks (Section V-D).

Post-decryption inference. After threshold decryption (Phase 3), weights become plaintext. Since weights are now plaintext constants, learnable operations (convolution, affine scaling) become plaintext-by-ciphertext multiplications, which do not consume multiplicative levels. Only polynomial activations, which involve ciphertext-by-ciphertext squaring, still consume levels.

2) Computational Cost: On the client side, the dominating cost is training the local teacher model, which is standard supervised learning and scales with the size of each client's dataset. Feature extraction requires a single forward pass through the trained teacher and is comparatively inexpensive.

On the server side, training proceeds block by block. For each block, the server performs encrypted gradient descent over pooled client data. The per-iteration cost is higher than plaintext training due to CKKS ciphertext operations, but the total number of iterations is modest because the block-level supervision signal is direct (input-output pairs rather than end-to-end labels).

3) Communication Cost: HE-IFD requires only a single communication round from each client to the server. The upload consists of encrypted input-output feature pairs for each block, plus per-channel normalisation statistics.

The dominant cost comes from encrypting intermediate feature maps, whose dimensionality far exceeds the normalisation statistics ($2C$ scalars) and evaluation keys (uploaded once per client). The total per-client upload volume is:

$$C_{\text{client}} = \sum_{k=0}^{K} \left\lceil \frac{D_k}{S} \right\rceil \times N_{\text{samples}} \times C_{\text{CT}} \tag{8}$$

where D_k is the feature dimensionality at block boundary k, S is the number of CKKS slots per ciphertext, N_{samples} is the number of samples each client uploads, and C_{CT} is the size of one ciphertext.

As a concrete example, in the ResNet-18 instantiation, Block 1 (the first residual layer group) produces feature maps of shape $64 \times 32 \times 32 = 65{,}536$ elements per sample. With 4096 CKKS slots per ciphertext (at ring dimension $\log \mathcal{N} = 13$), a single sample requires 16 ciphertexts per block boundary. Using 10% of a 5000-sample local dataset gives 500 samples per client, amounting to 8,000 ciphertexts for Block 1 alone. At approximately 0.6 MB per ciphertext, this gives roughly 4.8 GB per client per block.

For transformer architectures using logit-level distillation, the communication cost depends on the output dimensionality and the number of samples. For ViT-B/32 on CIFAR-10 (10 classes), each client uploads encrypted logits for 5000 samples, which is modest ($\sim$0.2 MB). For language models with large vocabularies, the cost is higher: a model with 50,000 output tokens and 5000 sequences of length 128 would produce a logit tensor of $\sim$48 GB in float32 or $\sim$24 GB in float16. Since logit-level distillation uses only a single "block" (the full model output), this is the total per-client upload. Note that the communication cost for multi-round encrypted FL [45], [6] would be orders of magnitude larger, as each of the hundreds of training rounds requires encrypting and uploading full model gradient updates from every client. Vanilla (unencrypted) FL avoids encryption overhead but provides no privacy protection. In both cases, the HE-IFD upload is performed once and requires no further interaction.

V. EXPERIMENTS

A. Setup

We evaluate HE-IFD on two image classification datasets (CIFAR-10 [53] and FashionMNIST [54]) with two model families (ResNet-18 and SimpleCNN), and extend the evaluation to pre-trained Vision Transformers (ViT-B/32). All accuracy experiments run as plaintext simulations of the encrypted protocol to expedite the evaluation: we execute the exact same forward and backward passes, loss functions, and weight updates that the CKKS-encrypted server would perform, and validate numerical equivalence against TenSEAL-based encrypted operations in Section V-F.

Data partitioning. Client data is split using a Dirichlet distribution with concentration parameter α, where a smaller α produces more heterogeneous partitions. We use

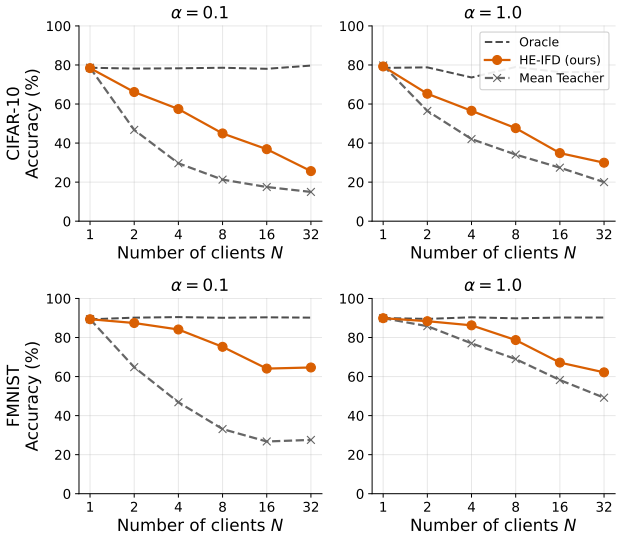

Fig. 2. HE-IFD accuracy on CIFAR-10 (top) and FashionMNIST (bottom) with ResNet-18 teachers and PolyResNet-18 student. The two plaintext reference lines are the centralised Oracle (dashed gray, upper bound trained on all pooled data with no privacy protection) and the mean individual teacher accuracy at each setting (solid gray, the natural local-only floor each client could achieve without collaboration). HE-IFD (orange) approaches the Oracle at small N and consistently outperforms the mean teacher across all settings.

$\alpha \in \{0.1, 1.0\}$ (extremely heterogeneous and heterogeneous) across $N \in \{1, 2, 4, 8, 16, 32\}$ clients.

ResNet-18. Each client trains a ResNet-18 teacher (11.2M parameters) for 200 epochs. The server distils into a PolyResNet-18 student with polynomial activations and per-channel affine scaling. Phase 2 trains each block independently on encrypted teacher features (80 epochs per block), then refines the cascade sequentially with bootstrapping to close the train-inference distribution gap. Each client contributes only 5000 samples (10% of CIFAR-10's 50,000 training images). This is a deliberate experiment choice, as intermediate feature distillation is data-efficient because teacher features provide direct per-block supervision, and the server trains on pooled features from all clients. Using a small fraction of each client's data keeps the upload volume manageable while achieving strong accuracy.

SimpleCNN. We train SimpleCNN teachers (0.3M parameters, 4 conv layers) and distil into an HE-compatible polynomial student.

Vision Transformer. We fine-tune pre-trained ViT-B/32 (ImageNet-1K, 88M parameters) on CIFAR-10 and FashionMNIST, replacing GELU with PolyGELU (degree-4) and LayerNorm with AffineNorm.

Reference points. We anchor every accuracy curve in this section to two plaintext references that bracket the achievable range without our protocol:

- **Oracle**: a standard ResNet-18 trained on all pooled data in plaintext, with no privacy protection. This is the centralised upper bound on accuracy and the strongest possible baseline a non-private system could achieve.
- **Mean individual teacher**: the unweighted average accuracy of the N local teacher models, each evaluated on the global test set after being trained only on its own private

partition. This is the strongest accuracy that any client could obtain on its own without collaboration, and it is the relevant comparator for assessing whether participating in the protocol is worthwhile.

We do not include a separate plaintext-KL distillation baseline: the question this paper addresses is not how polynomial activations compare with ReLU under standard distillation (a centralised question, already studied in [41], [9], [10]), but whether a one-shot encrypted federated protocol can recover an accuracy that lies between the local-only mean teacher and the centralised Oracle.

In the rest of this section, **HE-IFD** refers to our method. For ResNet-18 experiments the student is PolyResNet-18, for SimpleCNN experiments, the student is HEStudentCNN, and for ViT experiments, the student is a ViT-B/32 with GELU replaced by PolyGELU and LayerNorm replaced by AffineNorm (Section IV-D2).

B. Accuracy Results

Figure 2 shows accuracy across all settings on CIFAR-10 and FashionMNIST datasets. We summarize the key takeaways below:

HE-IFD consistently exceeds the mean teacher. Across all settings with $\alpha \leq 1.0$, the encrypted student outperforms the average individual teacher by 10–20 percentage points. The student aggregates complementary knowledge from heterogeneous specialists into a single model that no individual client could construct from its own data alone.

The effect of encryption remains negligible. On CIFAR-10, HE-IFD reaches 79.2% at N=1 while Oracle achieves 78.5%. On FashionMNIST, it reaches 89.9% at N=1, and the Oracle achieves 90.0%. Thus, the encrypted student achieves accuracy comparable to a centralised model trained on all data in plaintext. This shows that the effect of encryption on accuracy, when compared with the centralised model, remains negligible. **HE-IFD tracks the centralised Oracle as N increases.** At N=4, α=0.1, HE-IFD achieves 57% on CIFAR-10 and 85% on FashionMNIST. At N=16, α=0.1, it reaches 35–37% and 65–67%, respectively. The Oracle, which has no privacy constraint and sees all pooled data, degrades along the same shape as the per-client partition shrinks; HE-IFD's curve runs roughly parallel to it across N. This confirms that the residual accuracy loss is attributable to data fragmentation across more clients (weaker individual teachers), not to the encryption.

We also evaluate HE-IFD on SimpleCNN and pre-trained ViT-B/32 to confirm the framework generalises beyond ResNet. Figure 3 shows results on both datasets for $\alpha \in \{0.1, 1.0\}$. We summarize our findings below:

SimpleCNN. HE-IFD follows a similar trend to the Oracle at small N and degrades with more clients, consistent with the ResNet-18 results.

Vision Transformer. Oracle fine-tuning reaches 92% on CIFAR-10 and 91% on FashionMNIST (shown as the dashed gray Oracle line in Figure 3). HE-IFD follows a similar trend, with accuracy closest to the Oracle at N=1 and declining at larger N. After threshold decryption (Phase 3), clients may further fine-tune the received model on local data using standard GELU and LayerNorm, improving accuracy on their local distributions.

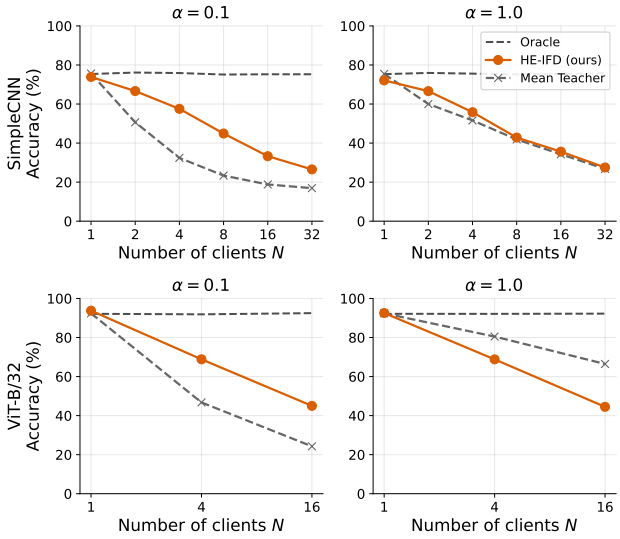

Fig. 3. Architecture generalisation on CIFAR-10. Top row: SimpleCNN for N in 1,2,4,8,16,32. Bottom row: pre-trained ViT-B/32 for N in 1,4,16 (fewer values because fine-tuning the 88M-parameter ViT teacher is computationally expensive). Left column: alpha=0.1, right column: alpha=1.0. The same trend from ResNet-18 holds across architectures: HE-IFD (orange) follows a similar trend to the Oracle across all settings.

C. Privacy Analysis

We compare HE-IFD against two differential privacy (DP) baselines on ResNet-18 CIFAR-10 at N=4, chosen so that the comparison spans both axes of the design space: the type of guarantee (statistical vs. cryptographic) and the number of communication rounds (one-shot vs. multi-round). Local Differential Privacy (LDP) [55] perturbs each client's data locally before upload and is therefore one-shot compatible; DP-SGD [56] injects calibrated Gaussian noise into per-sample gradients during multi-round training. Both yield a statistical (ε, δ)-guarantee parameterised by the privacy budget ε, with smaller ε corresponding to stronger privacy but more noise and lower utility. HE-IFD provides a cryptographic guarantee under the IND-CPA security of multiparty CKKS (Section IV-A3): the server's view consists of ciphertexts under a key whose corresponding secret is held in shares across the clients, and any inference about the inputs of an honest client reduces to breaking RLWE. Among the three, HE-IFD is the only one-shot mechanism with a cryptographic guarantee. Figure 4 shows the privacy-utility trade-off; HE-IFD appears as a single point at MIA AUC = 0.5 because the AUC is 0.5 by construction during training, independent of attack strength. Key findings:

HE-IFD matches the best DP-SGD accuracy with stronger privacy. At N=4, α=0.1, HE-IFD reaches 57.4% while the best DP-SGD (ε=10, weakest privacy) reaches 64.2%. At ε=1, DP-SGD drops to 33%. At ε=0.01, training collapses to 10%. HE-IFD achieves competitive accuracy without any noise injection and with a cryptographic rather than statistical privacy guarantee.

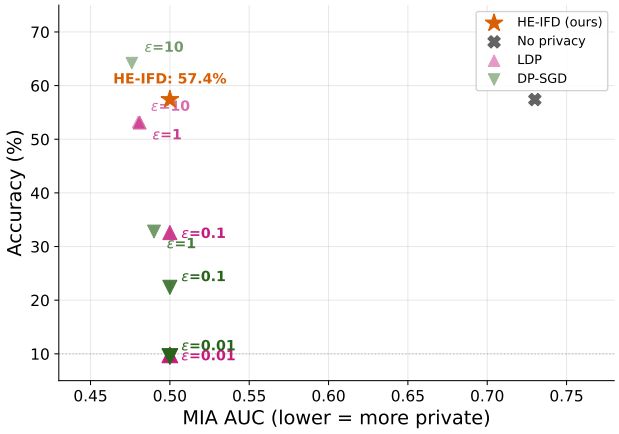

Fig. 4. Privacy-utility trade-off for ResNet-18 on CIFAR-10 (N=4). HE-IFD (orange point) provides accuracy without an attack surface; by ensuring the server never sees plaintext, it makes membership inference impossible during training (MIA AUC=0.5). LDP and DP-SGD points are shown with ε labels; darker markers indicate tighter privacy. At ε=0.01, both collapse to random-guess accuracy (∼10%) while still providing only a statistical guarantee. HE-IFD achieves comparable or better accuracy with a strictly stronger, cryptographic guarantee.

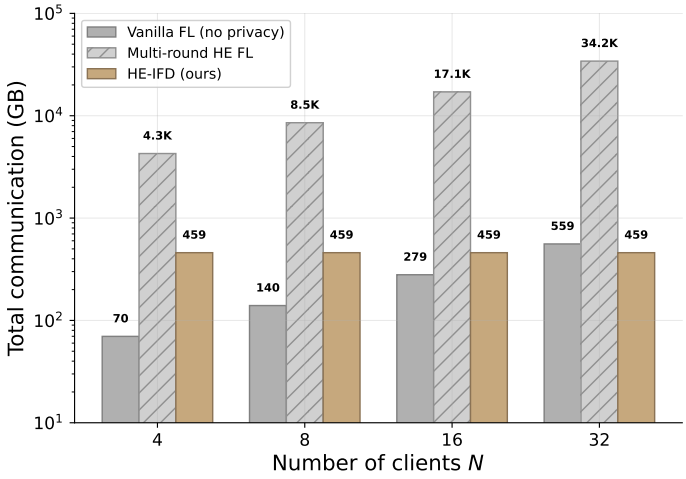

Fig. 5. Total communication for ResNet-18 on CIFAR-10. Multi-round methods (vanilla FL and encrypted FL with 200 rounds) scale linearly with N. HE-IFD (gold) requires a one-shot upload whose total is approximately constant (∼460 GB) regardless of N, and falls below unencrypted vanilla FL at N=32.

Membership inference confirms the cryptographic advantage. We evaluate LiRA [29] membership inference attacks with 8 shadow models, alongside loss-threshold [57] and confidence-based [58] attacks. Against the plaintext distillation student, LiRA achieves AUC 0.73, confirming high privacy risk from unencrypted logit transfers. HE-IFD eliminates this attack surface entirely: the server processes only ciphertexts, making membership inference computationally infeasible during training.

Client-to-client privacy after model release. After threshold decryption (Phase 3), each client holds the trained model in plaintext. At this point, standard membership inference risks apply to the released model, as with any machine learning model. However, the model was trained on pooled encrypted features from all N clients, so each individual client's contribution is diluted across the full training set. LiRA applied by one client against another requires shadow models trained on overlapping data, which is not possible when each client holds only its own private partition. The practical client-to-client inference risk is therefore limited to what can be extracted from the released model's predictions, comparable to any published/deployed machine learning model.

D. Communication Cost

HE-IFD requires a single upload per client. Each client encrypts teacher feature pairs under CKKS and uploads them along with evaluation keys. Clients upload only 10% of their local data, which is sufficient for high-accuracy distillation because intermediate features provide direct per-block supervision. This upload happens once, and no further client-server interaction occurs.

Figure 5 compares total communication across three regimes that span the design space introduced in Section II-E: (i) Vanilla (unencrypted) FedAvg [11], the multi-round plaintext baseline that establishes the linear-in-N floor; (ii) Multi-round encrypted FL, represented by POSEIDON [45] and BatchCrypt [6] (and architecturally similar systems such as FedSHE [47], FedML-HE [7], and CURE [8]), where clients encrypt gradient updates and upload them in every round; and (iii) HE-IFD, which encrypts teacher features instead of gradients and uploads them once. The axis of comparison is the unit that is encrypted (gradient vs. feature) and the number of rounds. In multi-round encrypted FL, each of 200 training rounds requires all N clients to encrypt their 11.2M-parameter gradient updates as CKKS ciphertexts (∼2.7 GB per client per round), upload them, and receive the aggregated model back. This accumulates to $\sim 1{,}068 \times N$ GB, reaching 4,272 GB at N=4 and 34 TB at N=32. Even unencrypted Vanilla FedAvg scales linearly with N, reaching 558 GB at N=32 over 200 rounds.

HE-IFD's total communication across all clients is approximately constant ($\sim 460GB$) regardless of N, because the total number of uploaded samples remains fixed. With N clients, each client's local partition contains $50{,}000/N$ training samples, of which 10% are contributed as encrypted features. The per-client upload therefore, decreases proportionally (∼115 GB at N=4, ∼15 GB at N=32), but the total across all clients remains fixed because the same training data is redistributed among more participants. At N=32, HE-IFD's total communication is lower than even unencrypted vanilla FL (460 vs. 558 GB), while providing cryptographic privacy.

E. Computation Cost

CKKS operates in SIMD mode (see Section III) where each ciphertext holds 8192 slots, and a single encrypted operation processes all slots simultaneously. With 5000 training samples, SIMD packing reduces the effective number of batches to approximately one per epoch. Phase 2a (block-level distillation, no bootstrapping) completes in approximately 0.6 CPU-hours. Phase 2b (cascade refinement with bootstrapping) takes approximately 5.5 CPU-hours, dominated by ∼900 bootstrap-

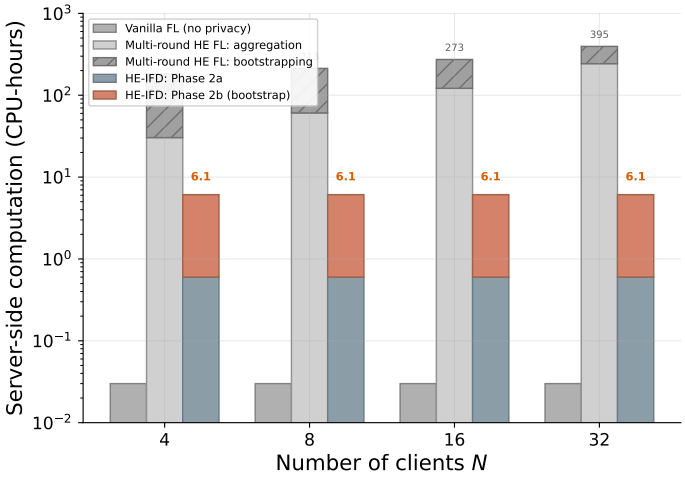

Fig. 6. Server-side computation for ResNet-18 on CIFAR-10. Vanilla FL server computation is negligible (weight averaging). Multi-round HE FL grows with N due to encrypted aggregation and periodic bootstrapping. HE-IFD requires $\sim$6.1 CPU-hours regardless of N, split between Phase 2a block-level distillation (steel blue) and Phase 2b cascade refinement with bootstrapping (coral).

ping operations at 20 seconds each. The total server-side computation is $\sim$6.1 CPU-hours, independent of N (Figure 6).

By comparison, the multi-round encrypted FL family discussed in Section II-E (POSEIDON, BatchCrypt, FedSHE) requires server-side encrypted aggregation that grows linearly with N: at N=4 this takes $\sim$7.6 CPU-hours for aggregation alone plus $\sim$152 hours of bootstrapping; at N=32, aggregation reaches $\sim$60 CPU-hours. The Oracle plaintext baseline trains in about 0.5 hours on a single GPU.

These timings reflect a single-threaded CPU baseline. Phase 2a blocks can be parallelised across cores. GPU-accelerated CKKS has demonstrated 10–100$\times$ speedups [59], bringing the total to under 1 GPU-hour.

F. Encrypted Arithmetic Feasibility

We validate that CKKS operations in our simulation match real encrypted computation using TenSEAL 0.3.16 with 128-bit security.

Numerical precision. At $\mathcal{N}$=16384 with 40-bit scale, relative error is below 10^{-5} per operation and below 10^{-3} for a complete training step. These errors are well below the noise floor of stochastic gradient descent.

SIMD packing. Each CKKS ciphertext holds $\mathcal{N}/2 = 8192$ slots. Feature maps smaller than 8192 elements (e.g., $512 \times 4 \times 4 = 8192$ at the deepest block, or 10-element logit vectors) allow multiple samples to share a single ciphertext. With 5000 training samples, SIMD packing reduces most block-boundary computations to one or two ciphertext operations per epoch, making the amortised per-sample cost negligible.

Multiplicative depth. During training, all parameters and data are ciphertexts. A single block training step requires approximately 12 levels, feasible with $\mathcal{N}$=32768 (18 levels at 128-bit security) without bootstrapping. Cascade refinement requires bootstrapping to refresh levels between blocks, with each bootstrap taking approximately 20 seconds per ciphertext.

Future directions. Recent work on stable polynomial network training [9], [10] suggests that improved activation initialisation may eliminate the need for cascade refinement, confining the entire protocol to the per-block depth budget. This would remove the bootstrapping requirement entirely and further optimize our framework.

VI. DISCUSSION AND CONCLUSION

We presented HE-IFD, a one-shot federated knowledge distillation protocol whose central property is that every client contribution and every server-side intermediate is, by construction, a homomorphic ciphertext under collectively held keys. The server's view of training is computationally indistinguishable from random under the IND-CPA security of multiparty CKKS, and release of the trained student requires the joint participation of all clients via collective key-switching. This is a fundamentally different privacy guarantee from the statistical one offered by differential privacy: it does not trade utility for privacy, and it does not require any assumption about the composition of noise across rounds.

Our experiments across three architectures (ResNet-18, SimpleCNN, and ViT-B/32) confirm that the framework generalises beyond a single model family. Compared against differential privacy baselines, HE-IFD achieves competitive accuracy with a strictly stronger privacy guarantee: at N=4, $\alpha = 0.1$, HE-IFD reaches 57.4% while the best DP-SGD configuration (epsilon=10, weakest privacy) achieves 64.2%, and tighter DP settings ($\varepsilon \leq 1$) collapse accuracy below 33%.

Our privacy analysis demonstrates that standard membership inference attacks (loss-threshold and confidence-based) achieve near-random AUC ($\sim$0.5) against the distilled student. As reviewed in Section II-B, stronger attacks from the literature, e.g., image reconstruction from logits, feature inversion, and advanced likelihood-ratio attacks, all require access to plaintext knowledge transfers, which HE eliminates entirely.

Future work includes reducing the one-shot upload cost ($\sim$115 GB per client at N=4, $\sim$ 460 GB total across all clients) through ciphertext compression and feature selection, integrating Byzantine-resilient aggregation under encryption, and investigating bootstrapping-free training schedules for deeper HE-compatible architectures.

ACKNOWLEDGMENTS

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude Sonnet 4.6 and Claude Opus 4.6 to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

REFERENCES

[1] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, vol. 32, 2019.

[2] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, vol. 37, no. 8, 2023, pp. 9925–9933.

[3] M. Jagielski, M. Nasr, K. Lee, C. Choquette-Choo, N. Carlini, and F. Tramèr, "Students parrot their teachers: Membership inference on model distillation," in *NeurIPS*, vol. 36, 2023.

[4] J. Shao, F. Wu, and J. Zhang, "Selective knowledge sharing for privacy-preserving federated distillation without a good teacher," *Nature Communications*, vol. 15, no. 1, pp. 1–15, 2024.

[5] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*. ACM, 2023, pp. 15–30.

[6] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning," in *USENIX ATC*. USENIX Association, 2020, pp. 493–506.

[7] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, "Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system," in *NeurIPS*, vol. 36, 2023.

[8] H. I. Kanpak, A. Shabbir, E. Genç, A. Küpçü, and S. Sav, "CURE: Privacy-preserving split learning done right," *arXiv preprint arXiv:2407.08977*, 2024.

[9] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 7, 2025.

[10] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.

[11] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[12] D. I. Dimitrov, M. Baader, and M. Vechev, "SPEAR: Exact gradient inversion of batches in federated learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/c13cd7feab4beb1a27981e19e2455916-Abstract-Conference.html

[13] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019, pp. 739–753.

[14] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017, pp. 603–618.

[15] R. Kerkouche, G. Mema, M. Fritz, J. Ramon, and N. Samarin, "Client-specific property inference against secure aggregation in federated learning," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2303.03908

[16] V. Carletti, P. Foggia, C. Mazzocca, G. Parrella, and M. Vento, "SoK: Gradient inversion attacks in federated learning," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/carletti

[17] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019. [Online]. Available: https://arxiv.org/abs/1910.03581

[18] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "DS-FL: Distillation-based semi-supervised federated learning for medical image classification," in *IEEE ICC*, 2021.

[19] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/18df51b97ccd68128e994804f3eccc87-Abstract.html

[20] H. Chang, V. Shejwalkar, R. Shokri, and A. Houmansadr, "Cronus: Robust and heterogeneous collaborative learning with black-box knowledge transfer," *arXiv preprint arXiv:1912.11279*, 2019.

[21] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 1322–1333. [Online]. Available: https://dl.acm.org/doi/10.1145/2810103.2813677

[22] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023, pp. 12 198–12 207.

[23] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023. [Online]. Available: https://dl.acm.org/doi/10.1016/j.jisa.2023.103475

[24] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *Proceedings on Privacy Enhancing Technologies*, vol. 2025, no. 4, pp. 201–215, 2025.

[25] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, ser. Lecture Notes in Computer Science, vol. 13423. Springer, 2023, pp. 364–378.

[26] X. Wang, L. Wu, and Z. Guan, "GradDiff: Gradient-based membership inference attacks against federated distillation with differential comparison," *Information Sciences*, vol. 658, p. 120068, 2024.

[27] X. Cui, H. Zhang, and Y. Pei, "On membership inference attacks in knowledge distillation," *arXiv preprint arXiv:2505.11837*, 2025.

[28] E. Galichin, M. Pautov, O. Zhavoronkin, O. Y. Rogov, and I. Oseledets, "GLiRA: Black-box membership inference attack via knowledge distillation," *IEEE Transactions on Information Forensics and Security*, 2025.

[29] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *IEEE S&P*, 2022, pp. 1897–1914. [Online]. Available: https://arxiv.org/abs/2112.03570

[30] J. Shao and J. Z. Fangzhao Wu, "Selective knowledge sharing for privacy-preserving federated distillation without a good public dataset," *Nature Communications*, vol. 15, p. 116, 2024. [Online]. Available: https://www.nature.com/articles/s41467-023-44383-9

[31] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019. [Online]. Available: https://arxiv.org/abs/1902.11175

[32] J. Wang *et al.*, "Towards one-shot federated learning: Advances, challenges, and future directions," *arXiv preprint arXiv:2505.02426*, 2025.

[33] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "Practical data-free federated learning," in *NeurIPS*, vol. 34, 2021, pp. 11 919–11 932.

[34] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.

[35] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.

[36] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021, pp. 1484–1490.

[37] L. Sun and L. Lyu, "Federated model distillation with noise-free differential privacy," in *IJCAI*, 2021, pp. 1563–1569.

[38] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025, pp. 2601–2610. [Online]. Available: https://openaccess.thecvf.com/content/WACV2025/html/Mendieta_Navigating_Heterogeneity_and_Privacy_in_One-Shot_Federated_Learning_with_Diffusion_Models_WACV_2025_paper.html

[39] Y. Xiong, R. Wang, M. Cheng, F. Yu, and C.-J. Hsieh, "Feddm: Iterative distribution matching for communication-efficient federated learning," in *CVPR*, 2023.

[40] G. Gad, E. Gad, Z. M. Fadlullah, M. M. Fouda, and N. Kato, "Communication-efficient and privacy-preserving federated learning via joint knowledge distillation and differential privacy in bandwidth-constrained networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 11, pp. 17 586–17 598, 2024.

[41] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkowich, "A methodology for training homomorphic encryption friendly neural networks," in *ACNS Workshops*, ser. Lecture Notes in Computer Science, vol. 13285. Springer, 2022, pp. 536–553.

[42] K. Garimella, N. K. Jha, and B. Reagen, "Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning," in *PPML Workshop, CCS*, 2021.

[43] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *Privacy in Machine Learning Workshop (PriML), NeurIPS*, 2021. [Online]. Available: https://www.eurecom.fr/en/publication/5626

[44] A. Pirillo and L. Colombo, "ReBoot: Encrypted training of deep neural networks with CKKS bootstrapping," 2025. [Online]. Available: https://arxiv.org/abs/2506.19693

[45] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.

[46] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *Proceedings on Privacy Enhancing Technologies*, vol. 2021, no. 4, pp. 291–311, 2021.

[47] X. Wei, R. Huang, L. Lei, and J. Wu, "Fedshe–cq: A communication-efficient homomorphic encryption framework for federated learning," *Computer Networks*, vol. 260, p. 111813, 2025.

[48] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017, pp. 409–437.

[49] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10820. Springer, 2018, pp. 360–384. [Online]. Available: https://eprint.iacr.org/2018/153
[50] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.
[51] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.
[52] M. Coşgün, M. Gençtürk, and S. Sav, "Bridging local and federated data normalization in federated learning: A privacy-preserving approach," *arXiv:2511.11249*, 2025.
[53] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.
[54] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1708.07747
[55] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" in *SIAM Journal on Computing*, vol. 40, no. 3, 2011, pp. 793–826.
[56] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 308–318. [Online]. Available: https://arxiv.org/abs/1607.00133
[57] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *31st IEEE Computer Security Foundations Symposium (CSF)*, 2018, pp. 268–282. [Online]. Available: https://arxiv.org/abs/1709.01604
[58] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: https://arxiv.org/abs/1806.01246
[59] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, and Y. Lee, "Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 4, pp. 114–148, 2021.

HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.

Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. Dr. Küpçü has various accomplishments including 8 international patents granted, 16 funded research projects (for 14 of which he was the principal investigator), 2 European Union COST Action management committee memberships, 4 Outstanding Teaching Awards, 4 Outstanding Young Scientist Awards, ACM and IEEE Senior Member Awards, and the Royal Society of UK Newton Advanced Fellowship. He is also a co-founder of Xtinge Technology Inc.

Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.